Error-based Learning

Chapter 7 – Complete Summary

Sections 7.1, 7.2, 7.3, 7.4, 7.5

Dr. Mohammed Brahimi & Dr. Aicha Boutorh

A clear and detailed summary of all concepts, formulas, and examples

Machine Learning Course

0 Contents

1 The Big Idea **2**

2 Fundamentals (Section 7.1) **2**
2.1 Simple Linear Regression . . . 2
2.1.1 The Running Example: Office Rentals . . . 2
2.1.2 The Line Equation . . . 2
2.2 Measuring Error . . . 3
2.2.1 The L2 Loss Function (Sum of Squared Errors) . . . 3
2.3 Error Surfaces . . . 4

3 Multivariate Linear Regression with Gradient Descent (Section 7.2) **4**
3.1 Multivariate Linear Regression . . . 4
3.2 Gradient Descent Algorithm . . . 5
3.2.1 Deriving the Weight Update Rule . . . 6
3.3 Choosing Learning Rates & Initial Weights . . . 6
3.4 A Complete Worked Example (Iteration by Iteration) . . . 6

4 Probabilistic View of Linear Regression (Section 7.3) **7**
4.1 Assumptions . . . 7
4.2 Maximum Likelihood Estimation (MLE) . . . 8
4.3 Maximum A Posteriori (MAP) and Ridge Regression . . . 8

5 Interpreting Multivariate Models (Section 7.4 – Part A) **8**
5.1 What Do the Weights Tell Us? . . . 8
5.2 Statistical Significance: The t-test . . . 9

6 Learning Rate Decay (Section 7.4 – Part B) **9**

7 Handling Categorical Features (Section 7.4 – Part C) **10**
7.1 One-Hot Encoding (Dummy Variables) . . . 10

8 Logistic Regression (Section 7.4 – Part D) **10**
8.1 The Problem with Hard Thresholds . . . 11
8.2 The Logistic (Sigmoid) Function . . . 11
8.3 Logistic Regression Model . . . 11
8.4 Loss Functions for Logistic Regression . . . 11
8.4.1 Squared Error Loss (not ideal) . . . 11
8.4.2 Binary Cross-Entropy Loss (preferred) . . . 12
8.5 Weight Update Rule for Logistic Regression . . . 12

9 Modeling Non-Linear Relationships (Section 7.5) **12**
9.1 Basis Functions . . . 12

10 Multinomial Logistic Regression (Section 7.5 – Multi-class) 13
10.1 One-vs-All Strategy 14
10.2 Normalising the Outputs (Softmax-like) 14

11 Support Vector Machines (SVMs) (Section 7.5) 14
11.1 The Maximum Margin Principle 15
11.2 The Decision Boundary (Hyperplane) 15
11.3 The SVM Model 15
11.4 Training Objective 15
11.5 Non-Linear SVMs and Kernel Functions 16

12 Chapter Summary 17

1 The Big Idea

Key Concept

Error-based learning follows three steps:

1. Start with a model with **random parameters** (weights).
2. Use an **error function** to measure how bad the predictions are.
3. **Iteratively adjust** the parameters to reduce the error and make better predictions.

Think of it like adjusting the temperature of a shower: you feel how hot/cold it is (error), then turn the knob a little (weight update), and repeat until the temperature is right (convergence).

2 Fundamentals (Section 7.1)

2.1 Simple Linear Regression

Simple linear regression models the relationship between **one input** (descriptive feature) and an **output** (target feature).

2.1.1 The Running Example: Office Rentals

We use a dataset of 10 Dublin offices. Each office has features like *Size*, *Floor*, *Broadband Rate*, and *Energy Rating*. The goal is to predict the **Rental Price**.

ID	Size (ft²)	Floor	Broadband	Energy	Rental Price
1	500	4	8	C	320
2	550	7	50	A	380
3	620	9	7	A	400
4	630	5	24	B	390
5	665	8	100	C	385
6	700	4	8	B	410
7	770	10	7	B	480
8	880	12	50	A	600
9	920	14	8	C	570
10	1000	9	24	B	620

2.1.2 The Line Equation

A simple linear model uses the equation of a straight line:

$$y = mx + b$$

In machine learning notation, we write the model as:

Formula

$$M_{\mathbf{w}}(\mathbf{d}) = w[0] + w[1] \times d[1]$$

where $w[0]$ is the **intercept** (bias), $w[1]$ is the **slope**, and $d[1]$ is the input feature (e.g. Size).

Worked Example

From a scatter plot of Size vs. Rental Price, we can see a clear linear relationship. The best-fit line found is:

$$\text{Rental Price} = 6.47 + 0.62 \times \text{Size}$$

To predict the price of a **730 ft² office**:

$$\text{Rental Price} = 6.47 + 0.62 \times 730 = \mathbf{459.07}$$

2.2 Measuring Error

With many possible lines, how do we find the *best* one? We need to measure how wrong our model is for each training instance.

For a single instance i, the **error** is:

$$\text{error}_i = t_i - M_{\mathbf{w}}(d_i[1])$$ error= true target - predicted value

where t_i is the true (target) value and $M_{\mathbf{w}}(d_i[1])$ is the predicted value.

2.2.1 The L2 Loss Function (Sum of Squared Errors)

We square the errors (to make them positive and penalise large errors more) and sum them up:

Formula

$$L_2(M_{\mathbf{w}}, D) = \frac{1}{2}\sum_{i=1}^{n}(t_i - M_{\mathbf{w}}(d_i[1]))^2 = \frac{1}{2}\sum_{i=1}^{n}\big(t_i - (w[0] + w[1] \times d_i[1])\big)^2$$

The $\frac{1}{2}$ factor is added to make the derivative cleaner later.

Calculating the SSE for the Office Rental Model

For the model Rental Price $= 6.47 + 0.62 \times$ Size:

ID	True Price	Predicted	Error	Squared Error
1	320	316.79	3.21	10.32
2	380	347.82	32.18	1035.62
3	400	391.26	8.74	76.32
4	390	397.47	−7.47	55.80
5	385	419.19	−34.19	1169.13
6	410	440.91	−30.91	955.73
7	480	484.36	−4.36	19.01
8	600	552.63	47.37	2243.90
9	570	577.46	−7.46	55.59
10	620	627.11	−7.11	50.51
	Sum of Squared Errors (Sum/2)			**2835.82**

A lower SSE means a better model.

2.3 Error Surfaces

For every combination of weights $(w[0], w[1])$, there is a corresponding SSE value. Plotting all these values creates an **error surface**.

Key Concept

- The x-y plane of weights is called the **weight space**.
- The surface plotted above it is called the **error surface**.
- The **lowest point** on the error surface corresponds to the **best model**.

Mathematically, the best model is found where the partial derivatives of the loss equal zero:

$$\frac{\partial}{\partial w[0]} \frac{1}{2} \sum_{i=1}^{n} (t_i - (w[0] + w[1] \cdot d_i[1]))^2 = 0$$

$$\frac{\partial}{\partial w[1]} \frac{1}{2} \sum_{i=1}^{n} (t_i - (w[0] + w[1] \cdot d_i[1]))^2 = 0$$

We solve this using a guided search method called **Gradient Descent**.

3 Multivariate Linear Regression with Gradient Descent (Section 7.2)

3.1 Multivariate Linear Regression

Instead of using only *one* feature (Size), we can use *multiple* features (Size, Floor, Broadband Rate, ...).

Formula

The general multivariate model:

$$M_{\mathbf{w}}(\mathbf{d}) = w[0] + w[1] \cdot d[1] + w[2] \cdot d[2] + \cdots + w[m] \cdot d[m] = w[0] + \sum_{j=1}^{m} w[j] \cdot d[j]$$

By defining a dummy feature $d[0] = 1$ always, this simplifies to the **dot product**:

$$M_{\mathbf{w}}(\mathbf{d}) = \sum_{j=0}^{m} w[j] \cdot d[j] = \mathbf{w} \cdot \mathbf{d}$$

The L2 loss function generalises naturally:

$$L_2(M_{\mathbf{w}}, D) = \frac{1}{2} \sum_{i=1}^{n} (t_i - (\mathbf{w} \cdot \mathbf{d}_i))^2$$

Worked Example

The multivariate model for office rentals using Size, Floor, and Broadband Rate:

$$\text{Rental Price} = w[0] + w[1] \times \text{Size} + w[2] \times \text{Floor} + w[3] \times \text{Broadband Rate}$$

After training, the weights are:

$$w[0] = -0.1513, \quad w[1] = 0.6270, \quad w[2] = -0.1781, \quad w[3] = 0.0714$$

To predict for a **690 ft²** office, **11th floor**, broadband **50 Mb/s**:

$$\text{Rental Price} = -0.1513 + 0.6270(690) - 0.1781(11) + 0.0714(50) = \mathbf{434.09}$$

3.2 Gradient Descent Algorithm

Gradient Descent is an iterative method to walk *downhill* on the error surface toward the minimum.

Algorithm 1 Gradient Descent for Training Linear Regression

Require: Training dataset D, learning rate α

```
w ← random starting point in weight space
repeat
    for all w[j] in w do
        w[j] ← w[j] + α × ERRORDELTA(D, w[j])
    end for
until convergence
```

3.2.1 Deriving the Weight Update Rule

For a single training instance (d, t), the gradient of the error surface with respect to weight $w[j]$ is:

$$\frac{\partial}{\partial w[j]} L_2 = (t - M_{\mathbf{w}}(\mathbf{d})) \times (-d[j])$$

Generalising to all n training instances:

Formula

$$\frac{\partial}{\partial w[j]} L_2(M_{\mathbf{w}}, D) = \sum_{i=1}^{n} \left((t_i - M_{\mathbf{w}}(\mathbf{d}_i)) \times (-d_i[j]) \right)$$

Therefore the **weight update rule** is:

$$w[j] \leftarrow w[j] + \alpha \sum_{i=1}^{n} \left((t_i - M_{\mathbf{w}}(\mathbf{d}_i)) \times d_i[j] \right)$$

where the sum is the **errorDelta** for weight $w[j]$.

Key Concept

Intuition: If our prediction is too low ($t_i > M_{\mathbf{w}}$), the error is positive. Adding a positive $\alpha \times$ errorDelta increases $w[j]$, which will raise future predictions. The algorithm self-corrects!

3.3 Choosing Learning Rates & Initial Weights

Important Note

The learning rate α controls the **step size** at each iteration.

- **Too small** ($\alpha \approx 0.002$): Very slow convergence, many iterations needed.
- **Just right** ($\alpha \approx 0.08$): Smooth, steady descent.
- **Too large** ($\alpha \approx 0.18$): Overshoots the minimum, may diverge.

Typical range: $\alpha \in [0.00001,\ 10]$. Use trial and error.
Initial weights: Randomly chosen from $[-0.2,\ 0.2]$ tends to work well.

3.4 A Complete Worked Example (Iteration by Iteration)

Office Rentals – Gradient Descent Trace

Setup: $\alpha = 0.00000002$, model uses Size, Floor, Broadband Rate.

Initial weights:

$$w[0] = -0.146, \quad w[1] = 0.185, \quad w[2] = -0.044, \quad w[3] = 0.119$$

Iteration 1: Compute predictions and errorDeltas for all 10 instances. The errorDeltas sum to:

$$\text{errorDelta}(w[0]) = 3185.61, \quad \text{errorDelta}(w[1]) = 2{,}412{,}073.9$$

$$\text{errorDelta}(w[2]) = 27{,}888.65, \quad \text{errorDelta}(w[3]) = 88{,}727.43$$

Sum of Squared Errors = 533,785.80

Update $w[1]$:

$$w[1] \leftarrow 0.185 + 0.00000002 \times 2{,}412{,}074 = \mathbf{0.2332}$$

New weights after Iteration 1:

$$w[0] = -0.146, \quad w[1] = 0.233, \quad w[2] = -0.043, \quad w[3] = 0.121$$

Iteration 2: SSE drops to 443,361.52. Weights continue improving.

After 100 iterations, SSE converges to 2,913.5 and weights stabilise at:

$$w[0] = -0.1513, \quad w[1] = 0.6270, \quad w[2] = -0.1781, \quad w[3] = 0.0714$$

4 Probabilistic View of Linear Regression (Section 7.3)

4.1 Assumptions

The probabilistic view gives us a deeper justification for using squared error.

Key Concept

Data assumption: $t_i \in \mathbb{R}$
Model assumption: The target is a linear function of the input plus Gaussian noise:

$$t_i = \mathbf{w} \cdot \mathbf{d}_i + \epsilon_i, \quad \epsilon_i \sim \mathcal{N}(0, \sigma^2)$$

This implies:

$$t_i \mid \mathbf{d}_i \sim \mathcal{N}(\mathbf{w} \cdot \mathbf{d}_i,\ \sigma^2)$$

So the probability of observing t_i given $\mathbf{d}_i$ and $\mathbf{w}$ is:

$$P(t_i \mid \mathbf{d}_i, \mathbf{w}) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(\mathbf{w} \cdot \mathbf{d}_i - t_i)^2}{2\sigma^2}\right)$$

Each data point is drawn from a Gaussian centered at $\mathbf{w} \cdot \mathbf{d}_i$.

4.2 Maximum Likelihood Estimation (MLE)

The goal is to find **w** that makes the observed data **most likely**. Assuming data points are independent:

$$\mathbf{w} = \arg\max_{\mathbf{w}} \prod_{i=1}^{n} P(t_i \mid \mathbf{d}_i, \mathbf{w})$$

Taking the log (which doesn't change the argmax) and simplifying:

$$\mathbf{w} = \arg\max_{\mathbf{w}} -\frac{1}{2\sigma^2} \sum_{i=1}^{n} (\mathbf{w} \cdot \mathbf{d}_i - t_i)^2$$

which is equivalent to:

Formula

$$\mathbf{w} = \arg\min_{\mathbf{w}} \frac{1}{2} \sum_{i=1}^{n} (\mathbf{w} \cdot \mathbf{d}_i - t_i)^2$$

Conclusion: Minimising the squared error loss is equivalent to maximising the likelihood of the data under a Gaussian noise assumption!

4.3 Maximum A Posteriori (MAP) and Ridge Regression

If we also consider a Gaussian prior on the weights $P(\mathbf{w}) \propto \exp(-\frac{\mathbf{w}^\top \mathbf{w}}{2\tau^2})$, the MAP estimate leads to:

Formula

$$\mathbf{w} = \arg\min_{\mathbf{w}} \frac{1}{2} \sum_{i=1}^{n} (\mathbf{w} \cdot \mathbf{d}_i - t_i)^2 + \lambda \|\mathbf{w}\|_2^2$$

This is called **Ridge Regression**. The term $\lambda \|\mathbf{w}\|_2^2$ is a **regularisation** penalty:

- It favours smaller weights.
- It prevents overfitting.
- The prior acts as our initial belief; the data shifts this belief to a posterior.

5 Interpreting Multivariate Models (Section 7.4 – Part A)

5.1 What Do the Weights Tell Us?

Each weight $w[j]$ indicates how much the prediction changes when feature $d[j]$ changes by 1 unit.

Feature	Weight	Std. Error	t-statistic	p-value
Size	0.6270	0.0545	11.504	< 0.0001
Floor	-0.1781	2.7042	-0.066	0.949
Broadband Rate	0.0714	0.2969	0.240	0.816

5.2 Statistical Significance: The t-test

Simply comparing weight magnitudes is **misleading** (features may have different scales). We use a **t-test** instead.

Formula

Step 1 – Standard error of the model:

$$se = \sqrt{\frac{\sum_{i=1}^{n}(t_i - M_{\mathbf{w}}(\mathbf{d}_i))^2}{n-2}}$$

Step 2 – Standard error for feature $d[j]$:

$$se(d[j]) = \frac{se}{\sqrt{\sum_{i=1}^{n}(d_i[j] - \bar{d}[j])^2}}$$

Step 3 – t-statistic:

$$t = \frac{w[j]}{se(d[j])}$$

Key Concept

Decision rule: If the p-value < 0.05, the feature is **statistically significant**. From the table above:

- **Size** has $p < 0.0001 \Rightarrow$ very significant predictor.
- **Floor** has $p = 0.949 \Rightarrow$ not significant; can be removed.
- **Broadband Rate** has $p = 0.816 \Rightarrow$ not significant.

6 Learning Rate Decay (Section 7.4 – Part B)

Key Concept

A fixed learning rate can be problematic:

- Large early: fast initial progress.
- Large late: overshooting the minimum.

Solution – Learning Rate Decay: Start large, then reduce over time.

Formula

$$\alpha_\tau = \alpha_0 \cdot \frac{c}{c+\tau}$$

- α_0: initial learning rate.
- c: constant controlling decay speed (larger c = slower decay).
- τ: current iteration number.

For example, with $\alpha_0 = 0.18$ and $c = 10$, the learning rate quickly shrinks as iterations increase, giving fast convergence without overshooting.

7 Handling Categorical Features (Section 7.4 – Part C)

Linear regression requires **numeric** features, but we often have categorical ones (e.g. Energy Rating = A, B, C).

7.1 One-Hot Encoding (Dummy Variables)

Convert a categorical feature with k levels into k binary (0/1) features.

Worked Example

Energy Rating (A, B, C) becomes three new features:

ID	Energy Rating	Rating_A	Rating_B	Rating_C
1	C	0	0	1
2	A	1	0	0
3	A	1	0	0
4	B	0	1	0

The extended regression model becomes:

$$\begin{aligned}\text{Rental Price} = w[0] &+ w[1] \times \text{Size} + w[2] \times \text{Floor} \\ &+ w[3] \times \text{Broadband} + w[4] \times \text{Rating_A} \\ &+ w[5] \times \text{Rating_B} + w[6] \times \text{Rating_C}\end{aligned}$$

8 Logistic Regression (Section 7.4 – Part D)

So far we predicted **continuous** values. Now we handle **categorical targets** (classification).

8.1 The Problem with Hard Thresholds

A simple step function:

$$M_{\mathbf{w}}(\mathbf{d}) = \begin{cases} 1 & \text{if } \mathbf{w} \cdot \mathbf{d} \geq 0 \\ 0 & \text{otherwise} \end{cases}$$

has two problems:

- It is **not differentiable** ⇒ can't compute gradient.
- It gives overconfident 0/1 predictions ⇒ no probability estimate.

8.2 The Logistic (Sigmoid) Function

Formula

$$\text{Logistic}(x) = \frac{1}{1 + e^{-x}}$$

Properties:

- Output is always in $(0, 1)$ ⇒ interpreted as a probability.
- Smooth and differentiable.
- $\text{Logistic}(0) = 0.5$; as $x \to +\infty$, output $\to 1$; as $x \to -\infty$, output $\to 0$.

8.3 Logistic Regression Model

Formula

$$M_{\mathbf{w}}(\mathbf{d}) = \text{Logistic}(\mathbf{w} \cdot \mathbf{d}) = \frac{1}{1 + e^{-\mathbf{w} \cdot \mathbf{d}}}$$

This gives a smooth "decision surface" instead of a hard boundary.

Worked Example

Generators Dataset: 56 generators labelled "good" or "faulty" based on RPM and Vibration.
The trained logistic regression model:

$$M_{\mathbf{w}}(\langle \text{RPM, Vibration} \rangle) = \frac{1}{1 + e^{-(-0.4077 + 4.1697 \times \text{RPM} + 6.0460 \times \text{Vibration})}}$$

Interpretation: Output ≈ 1 means "faulty", output ≈ 0 means "good".

8.4 Loss Functions for Logistic Regression

8.4.1 Squared Error Loss (not ideal)

$$L_{SE} = \sum_i (t_i - M_{\mathbf{w}}(\mathbf{d}_i))^2$$

Problem: The extra factor $M_{\mathbf{w}}(1 - M_{\mathbf{w}})$ in the gradient slows learning near 0 or 1.

8.4.2 Binary Cross-Entropy Loss (preferred)

Formula

$$L_{BCE} = -\sum_i \left[t_i \log M_{\mathbf{w}}(\mathbf{d}_i) + (1 - t_i) \log(1 - M_{\mathbf{w}}(\mathbf{d}_i)) \right]$$

Gradient:

$$\frac{\partial L_{BCE}}{\partial w} = \sum_i (M_{\mathbf{w}}(\mathbf{d}_i) - t_i)\, \mathbf{d}_i$$

This is clean – **no extra slowing factor**. Training is faster and more stable.

8.5 Weight Update Rule for Logistic Regression

Formula

$$w[j] \leftarrow w[j] + \alpha \sum_{i=1}^{n} \big((t_i - M_{\mathbf{w}}(\mathbf{d}_i)) \times M_{\mathbf{w}}(\mathbf{d}_i) \times (1 - M_{\mathbf{w}}(\mathbf{d}_i)) \times d_i[j] \big)$$

This is almost the same as for linear regression, with one extra factor: $M_{\mathbf{w}}(\mathbf{d}_i) \times (1 - M_{\mathbf{w}}(\mathbf{d}_i))$.

Important Note

Always **normalise** descriptive features before training logistic regression (e.g. to the range $[-1, 1]$). This prevents one feature from dominating the gradient updates.

9 Modeling Non-Linear Relationships (Section 7.5)

Real-world data is often not linearly related. We handle this by transforming the **input features** using **basis functions**.

9.1 Basis Functions

Formula

$$M_{\mathbf{w}}(\mathbf{d}) = \sum_{k=0}^{b} w[k] \times \phi_k(\mathbf{d})$$

where $\phi_0, \phi_1, \ldots, \phi_b$ are **basis functions** that transform the input in different ways.

Key Concept

Key idea: We do *not* change the model structure – we change how we represent the data. This means gradient descent still works exactly the same way!

Grass Growth Dataset – Polynomial Basis

Problem: Predict grass Growth from Rainfall. The relationship is non-linear (too little or too much rain hurts growth).
Basis functions for a quadratic model:

$$\phi_0(\text{Rain}) = 1, \quad \phi_1(\text{Rain}) = \text{Rain}, \quad \phi_2(\text{Rain}) = \text{Rain}^2$$

Model:

$$\text{Growth} = w[0] \cdot \phi_0 + w[1] \cdot \phi_1 + w[2] \cdot \phi_2$$

After training:

$$\text{Growth} = 0.3707 - 1.717 \times \text{Rain}^2 + 0.8475 \times \text{Rain}$$

This is a parabola – perfectly capturing the non-linear relationship.

Basis functions also extend to:

- **Multivariate** models: apply multiple basis functions to multiple features.
- **Logistic regression**: use the logistic function after the basis expansion.

EEG Dataset – Non-linear Classification

For classifying brain responses (positive/negative images) from EEG signals (P20 and P45), linear logistic regression fails. Using polynomial basis functions up to degree 3:

$$\phi_0 = 1, \ \phi_1 = \text{P20}, \ \phi_2 = \text{P45}, \ \phi_3 = \text{P20}^2, \ \phi_4 = \text{P45}^2,$$
$$\phi_5 = \text{P20}^3, \ \phi_6 = \text{P45}^3, \ \phi_7 = \text{P20} \times \text{P45}$$

This creates a curved decision boundary that correctly separates the two classes.

10 Multinomial Logistic Regression (Section 7.5 – Multi-class)

When the target has **more than 2 levels**, we extend logistic regression using a **one-versus-all** approach.

10.1 One-vs-All Strategy

For r target levels, train r separate binary logistic regression models:

$$\begin{aligned} M_{\mathbf{w}_1}(\mathbf{d}) &= \text{Logistic}(\mathbf{w}_1 \cdot \mathbf{d}) \quad \text{(level 1 vs. all others)} \\ M_{\mathbf{w}_2}(\mathbf{d}) &= \text{Logistic}(\mathbf{w}_2 \cdot \mathbf{d}) \quad \text{(level 2 vs. all others)} \\ &\vdots \\ M_{\mathbf{w}_r}(\mathbf{d}) &= \text{Logistic}(\mathbf{w}_r \cdot \mathbf{d}) \quad \text{(level } r \text{ vs. all others)} \end{aligned}$$

10.2 Normalising the Outputs (Softmax-like)

The raw outputs do not sum to 1. We normalise:

Formula

$$M'_{\mathbf{w}_k}(\mathbf{d}) = \frac{M_{\mathbf{w}_k}(\mathbf{d})}{\sum_{l \in \text{levels}(t)} M_{\mathbf{w}_l}(\mathbf{d})}$$

Final prediction:

$$M(\mathbf{q}) = \arg \max_{l \in \text{levels}(t)} M'_{\mathbf{w}_l}(\mathbf{q})$$

Customer Type Dataset (single / family / business)

Three models are trained. For a query with SPEND $= -0.7279$, FREQ $= 0.4789$:

$$M_{\mathbf{w}_{\text{single}}} = 0.9999, \quad M_{\mathbf{w}_{\text{family}}} = 0.0128, \quad M_{\mathbf{w}_{\text{business}}} = 0.0518$$

Total $= 1.0645$. Normalised:

$$M'_{\text{single}} = \frac{0.9999}{1.0645} \approx 0.9393 \quad \Leftarrow \text{highest}$$

Prediction: "single"

11 Support Vector Machines (SVMs) (Section 7.5)

SVMs are a powerful alternative to logistic regression for classification. Instead of just finding *any* decision boundary, they find the **best** one.

11.1 The Maximum Margin Principle

Key Concept

Key idea: Among all decision boundaries that correctly separate the classes, choose the one with the **largest margin** (widest gap between the two classes).
A large margin means the model generalises better to new data.

The instances that lie exactly on the margin boundaries are called **support vectors** – they define the decision boundary.

11.2 The Decision Boundary (Hyperplane)

The separating hyperplane is defined as:

$$w_0 + \mathbf{w} \cdot \mathbf{d} = 0$$

The SVM labels:

- Positive class: $w_0 + \mathbf{w} \cdot \mathbf{d} \geq +1$
- Negative class: $w_0 + \mathbf{w} \cdot \mathbf{d} \leq -1$

11.3 The SVM Model

Formula

$$M_{\boldsymbol{\alpha},w_0}(\mathbf{q}) = \sum_{i=1}^{s} t_i \cdot \alpha[i] \cdot (\mathbf{d}_i \cdot \mathbf{q}) + w_0$$

where:

- $(\mathbf{d}_1, t_1), \ldots, (\mathbf{d}_s, t_s)$ are the s **support vectors**.
- $\alpha[1], \ldots, \alpha[s]$ are **Lagrange multipliers** learned during training.
- w_0 is the bias weight.

Predictions: output $\leq -1 \Rightarrow$ negative class; output $\geq +1 \Rightarrow$ positive class.

11.4 Training Objective

The margin size is $\frac{2}{\|\mathbf{w}\|}$. To maximise the margin:

Formula

$$\text{Maximise } \frac{2}{\|\mathbf{w}\|} \quad \Longleftrightarrow \quad \text{Minimise } \|\mathbf{w}\|$$

Subject to the constraint:

$$t_i \times (w_0 + \mathbf{w} \cdot \mathbf{d}_i) \geq 1 \quad \forall i$$

This is a **quadratic optimisation** problem.

Worked Example

For the generators dataset, the support vectors are:

$$(\langle -0.225,\ 0.217\rangle,\ +1),\quad (\langle -0.066,\ -0.069\rangle,\ -1),\quad (\langle -0.273,\ -0.080\rangle,\ -1)$$

with $w_0 = -0.1838$ and $\boldsymbol{\alpha} = \langle 22.056,\ 6.998,\ 16.058\rangle$.
For query $\mathbf{q}_1 = \langle -0.314,\ -0.251\rangle$:

$$M_{\boldsymbol{\alpha},w_0}(\mathbf{q}_1) = -2.145 \leq -1 \Rightarrow \textbf{faulty generator}$$

For query $\mathbf{q}_2 = \langle -0.117,\ 0.31\rangle$:

$$M_{\boldsymbol{\alpha},w_0}(\mathbf{q}_2) = +1.592 \geq +1 \Rightarrow \textbf{good generator}$$

11.5 Non-Linear SVMs and Kernel Functions

When the data is **not linearly separable**, we use basis functions to project data into a higher-dimensional space where it *is* linearly separable.

Key Concept

Kernel Trick: Instead of explicitly computing basis functions $\phi(\mathbf{d})$ (expensive), we use a **kernel function** that gives the same result:

$$\phi(\mathbf{d}_i) \cdot \phi(\mathbf{q}) = \text{kernel}(\mathbf{d}_i, \mathbf{q})$$

Much cheaper computationally!

Formula

SVM with Kernel:

$$M_{\boldsymbol{\alpha},\text{kernel},w_0}(\mathbf{q}) = \sum_{i=1}^{s} t_i \cdot \alpha[i] \cdot \text{kernel}(\mathbf{d}_i, \mathbf{q}) + w_0$$

Common kernels:

Kernel	Formula
Linear	$\text{kernel}(\mathbf{d}, \mathbf{q}) = \mathbf{d} \cdot \mathbf{q} + c$
Polynomial	$\text{kernel}(\mathbf{d}, \mathbf{q}) = (\mathbf{d} \cdot \mathbf{q} + 1)^p$
Gaussian RBF	$\text{kernel}(\mathbf{d}, \mathbf{q}) = \exp(-\gamma \lVert \mathbf{d} - \mathbf{q} \rVert^2)$

12 Chapter Summary

Complete Chapter Overview

Section 7.1

Fundamentals: Linear regression, the L2 loss, error surfaces. We measure how bad our model is using the sum of squared errors and visualise this as a surface in weight space.

Section 7.2

Gradient Descent: Iteratively walk downhill on the error surface by updating each weight using the *errorDelta*. Choose α carefully. Use random initial weights in $[-0.2, 0.2]$.

Section 7.3

Probabilistic View: Minimising squared error = maximising likelihood under Gaussian noise. Adding a Gaussian prior on weights gives Ridge Regression (regularisation).

Section 7.4

Extensions: Interpret weights using t-statistics. Use learning rate decay for better convergence. Handle categorical features with one-hot encoding. Use logistic regression for binary classification (smooth sigmoid, cross-entropy loss).

Section 7.5

Advanced Topics: Basis functions for non-linear relationships. Multinomial logistic regression for multi-class problems (one-vs-all + normalisation). SVMs for maximum-margin classification (support vectors + kernel trick).

These notes are based on the course slides by Dr. Mohammed Brahimi and Dr. Aicha Boutorh.